



IS231: Web Technology Internet Protocols



By: Neamat El Tazi

References

Books:

- **Web Application Architecture: Principles, protocols and practices By Leon Shklar and Richard Rosen (Chapter 2)**

Images and slides References:

- <https://www.guru99.com/tcp-ip-model.html>
- https://www.slideshare.net/mario_cardinal/http-fundamentals-for-developers

TCP/IP Layers

The TCP/IP transport layer protocols ensure that packets arrive in sequence and without error, by swapping acknowledgments of data reception, and retransmitting lost packets. This type of communication is known as “end-to-end.” Transport layer protocols

Application

- To allow access to network resources

Transport

- To provide reliable process to process message delivery and error delivery

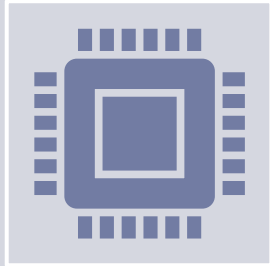
Internet

- To move packets from source to destination
- To provide internetworking

Network Interface

Responsible for the transmission for the between two device on the same network.

TCP/IP Protocols



TCP:

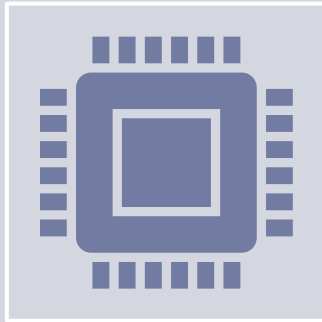
Transmission Control Protocol is an internet protocol suite which breaks up the message into TCP Segments and reassembling them at the receiving side.



IP:

An Internet Protocol address that is also known as an IP address is a numerical label. It is assigned to each device that is connected to a computer network which uses the IP for communication. Its routing function allows internetworking and essentially establishes the Internet. Combination of IP with a TCP allows developing a virtual connection between a destination and a source.

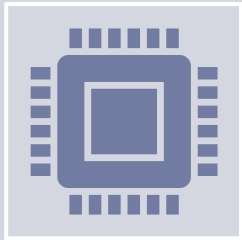
TCP/IP Protocols



HTTP:

The Hypertext Transfer Protocol is a foundation of the World Wide Web. It is used for transferring webpages and other such resources from the HTTP server or web server to the web client or the HTTP client. Whenever you use a web browser like Google Chrome or Firefox, you are using a web client.

TCP/IP Protocols



SMTP:

SMTP stands for Simple mail transfer protocol. This protocol supports the e-mail is known as a simple mail transfer protocol. This protocol helps you to send the data to another e-mail address.

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. Caching
6. Cookies
7. Connection
8. Security

1. Resources (URL): Locating resources using Universal Resource Locator

← → ↻ **linkedin.com/in/neamateltazi/** ↗ ☆

The Open Source D... Performance and T... Free Learning | PAC... competitive-landsc... Implemetation of 1... VL DB Library

in Search Home My Network Jobs Messaging

 **Empowering Companies**
LEVERAGING LATEST TECHNOLOGIES

Neamat ElTazi 🔊
Co-Founder and COO At AbuErdan| Passionate about Digital Transformation in Multiple Industries | Bridging the gap between Academia and Industry
Talks about #poultry, #innovation, #entrepreneurs, #cloudbasedsoftware, and #precisionlivestock
Egypt · [Contact info](#)
7,690 followers · 500+ connections

 AbuErdan
(www.abuerdan.com)
 Cairo University and University of Michigan

1. Resources (URL)

- ▶ `<scheme>://<host>:<port>/<path>?<query>#<fragment>`
`https://www.linkedin.com:443/in/neamateltazi/`
- ▶ URL Scheme: https (other cases http, ftp)
- ▶ Host: www.linkedin.com
- ▶ Port: 443 (default for Htpps) (Note 80 is the default for http)
- ▶ URL path: /in/neamateltazi

1. Resources (URL) – Cont.

▶ `<scheme>://<host>:<port>/<path>?<query>#<fragment>`

`http://www.google.com/search?q=computers`

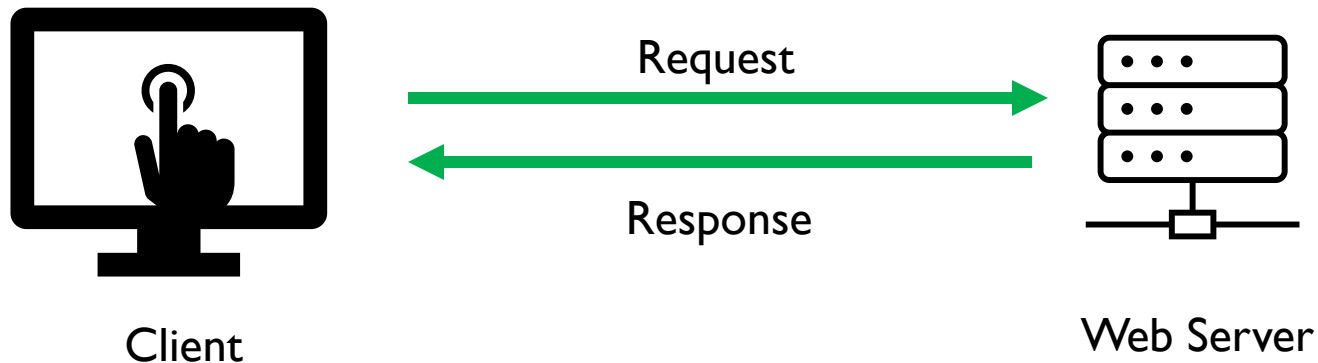
- ▶ URL Scheme: http
- ▶ Host: www.google.com
- ▶ Port: 80 (default value)
- ▶ URL path: /search
- ▶ Query string: ?q=computers

Fundamentals of HTTP

1. Resources
2. **Request**
3. Response
4. Media Type
5. Caching
6. Cookies
7. Connection
8. Security

HTTP Request and Response

- ▶ A client sends an HTTP request to a server using a message (header and body)
- ▶ A server responds by sending an HTTP response.
- ▶ The request and the response are two different message types.



2. Request Message



`[method][URL][version]`



`[headers]`



`[body]`

2. HTTP Request Methods

Method	Meaning
GET	Read data
POST	Insert data
PUT or PATCH	Update data, or insert if a new id
DELETE	Delete data

2. Request Message



[method][URL][version]

Method: Get
URL: <http://www.google.com>
Version: HTTP/1.1



[headers]

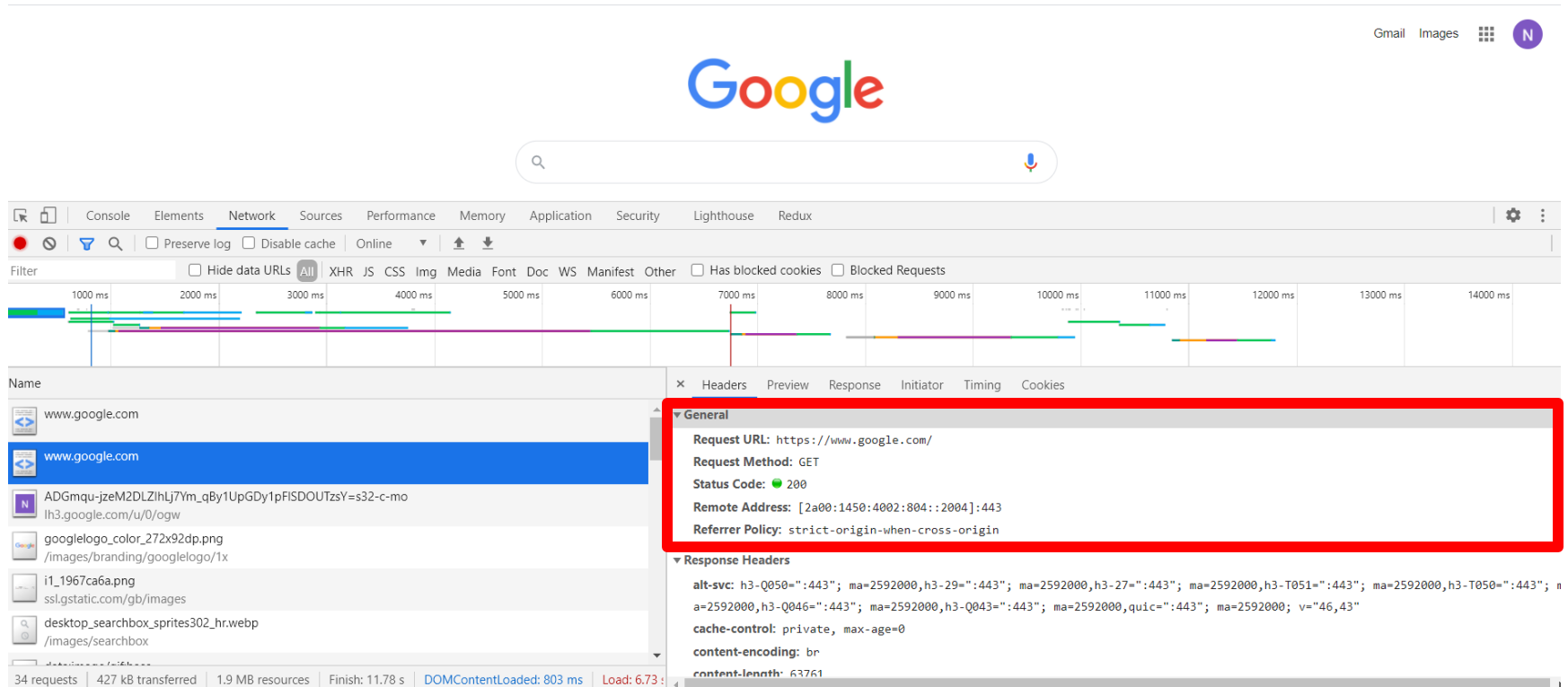


[body]

2. HTTP Request Headers

Header	Description
Referer	When the user clicks on a link, the client can send the URL of the referring page in this header
User-Agent	Information about the user agent (the software) making the request. Many application use the information in this header, when present, to know which browser is making the request (Chrome or Firefox or Internet Explorer or Safari)
Accept	Describes the media types (nature or format of a document, file or data) the user agent is willing to accept.
Accept-Language	Describes the languages the user agent prefers
Cookie	Cookie information generally helps a server track or identify a user
If-Modified-Since	Will contain a date of when the user agent last retrieved the resource. The server only has to send back the entire resource if it's been modified since that time

2. HTTP Request Headers (Browser inspect tool)



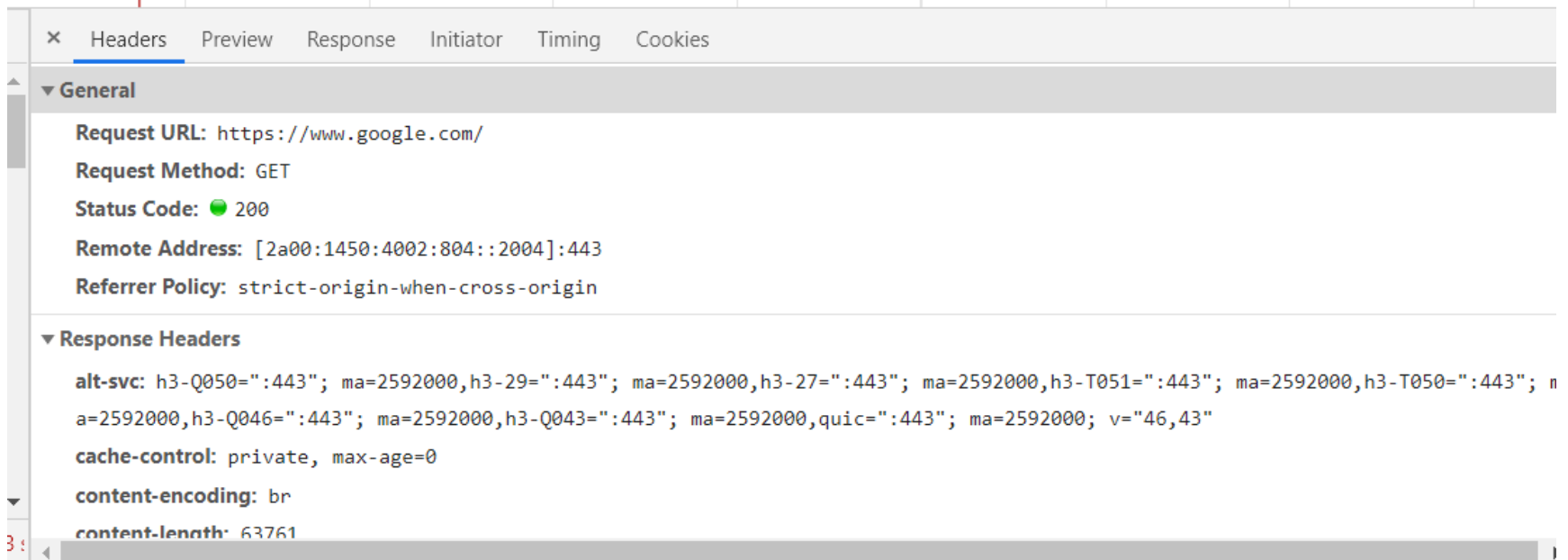
The screenshot shows a web browser with the Google homepage. The developer tools network tab is open, displaying a list of requests. The first request, 'www.google.com', is selected. The 'Headers' tab is active, showing the following information:

- Request URL:** https://www.google.com/
- Request Method:** GET
- Status Code:** 200
- Remote Address:** [2a00:1450:4002:804::2004]:443
- Referrer Policy:** strict-origin-when-cross-origin

The 'Response Headers' tab is also visible, showing the following information:

- alt-svc:** h3-Q050=":443"; ma=2592000, h3-29=":443"; ma=2592000, h3-27=":443"; ma=2592000, h3-T051=":443"; ma=2592000, h3-T050=":443"; ma=2592000, h3-Q046=":443"; ma=2592000, h3-Q043=":443"; ma=2592000, quic=":443"; ma=2592000; v="46,43"
- cache-control:** private, max-age=0
- content-encoding:** br
- content-length:** 63761

2. HTTP Request Headers (Browser inspect tool)



2. HTTP Request Message Example with Body

```
<form action="/account/create" method="POST">  
  <label for="firstName">First name</label>  
  <input id="firstName" name="firstName" type="text" />  
  <label for="lastName">Last name</label>  
  <input id="lastName" name="lastName" type="text" />  
  <input type="submit" value="Sign up!" />  
</form>
```

```
POST  
http://server.com:1060/account/create  
HTTP/1.1
```

```
Host: server.com
```

```
firstName=Mario&lastName=Cardinal
```

Fundamentals of HTTP

1. Resources
2. Request
3. **Response**
4. Media Type
5. Caching
6. Cookies
7. Connection
8. Security

3. HTTP Response Message



[version] [status] [reason]



[headers]



[body]

3. HTTP Response Status Codes

Range	Category
100 – 199	Informational 100 Continue
200 – 299	Successful 200 OK 201 Created 204 No Content
300 – 399	Redirection 301 Moved Permanently 304 Not Modified
400 – 499	Client Error 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found
500 – 599	Server Error 500 Internal Server Error 503 Service Unavailable

3. HTTP Response Message



[version] [status] [reason]

HTTP/1.1
200
OK



[headers]



[body]

3. HTTP Response Header

Header	Description
Connection	Options that are desired for the connection
Content-Encoding	The type of encoding used on the data
Content-Length	The length of the response body in octets (8-bit bytes)
Content-Type	Describes the media type of this content
Date	The date and time that the message was sent
Expires	Gives the date/time after which the response is considered stale.
Location	Used in redirection, or when a new resource has been created
Server	A name of the server

3. HTTP Response Message



[version] [status] [reason]

HTTP/1.1
200
OK



[headers]

Content-Type: text/html; charset=utf-8
Server: Microsoft-IIS/7.0
X-AspNet-Version: 2.0.50727
X-Powered-By: ASP.NET
Date: Sat, 14 Jan 2012 04:00:08 GMT
Connection: close
Content-Length: 17151



[body]

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. **Media Type**
5. Caching
6. Cookies
7. Connection
8. Security

4. Resources and Media Types

- ▶ When a host responds to an HTTP request, it returns a resource (content)
- ▶ Host also specifies the content type (also known as the media type) of the resource
 - ▶ Defined using Multipurpose Internet Mail Extensions (MIME)
 - ▶ “text/html”
 - ▶ “image/jpeg”
 - ▶ “text/xml”
 - ▶ “application/json”

4. Content Negotiation

- ▶ Request message

- ▶ Accept: text/html, application/xhtml+xml, application/xml;

- ▶ Response message

- ▶ Content-Type: text/html; charset=utf-8

HTTP Response Message



[version] [status] [reason]

HTTP/1.1
200
OK



[headers]

Content-Type: text/html; charset=utf-8
Server: Microsoft-IIS/7.0
X-AspNet-Version: 2.0.50727
X-Powered-By: ASP.NET
Date: Sat, 14 Jan 2012 04:00:08 GMT
Connection: close
Content-Length: 17151



[body]

```
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    ... content ...
  </body>
</html>
```

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. **Caching**
6. Cookies
7. Connection
8. Security

5 .Time-Based Caching

HTTP/1.1 200 OK

Last-Modified: Wed, 25 Jan 2012 17:55:15 GMT

Expires: Sat, 22 Jan 2022 17:55:15 GMT

Cache-Control: max-age=315360000,public

Content-Length: 208

<html>

<head>

</head>

<body>

</body>

</html>

5. Content-Based Caching

HTTP/1.1 200 OK

Last-Modified: Fri, 06 Jan 2012 18:08:20 GMT

ETag: "8e5bcd-59f-4b5dfef104d00"

Content-Type: text/xml

Vary: Accept-Encoding

Content-Encoding: gzip

Content-Length: 437

<html>

<head> >

</head>

<body>

</body>

</html>

5. HTTP Request and Caching

Request

GET ... HTTP/1.1

If-Modified-Since: Wed, 25 Jan 2012 17:55:15 GMT

Response

HTTP/1.1 **304 Not Modified**

Expires: Sat, 22 Jan 2022 17:16:19 GMT

Cache-Control: max-age=315360000,public

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. Caching
- 6. Cookies**
7. Connection
8. Security

6. Cookies

HTTP/1.1 200 OK

Content-Type: text/html; charset=utf-8

Set-Cookie: fname=Mario\$lname=Cardinal;
expires=Monday, 09-July-2012 21:12:00 GMT
domain=.mywebsite.com; path=/ ; HttpOnly

6. Identification and Cookies

- There is a size limitation of 4 KB
- Many websites only put in a unique identifier for a user

HTTP/1.1 200 OK

Set-Cookie:

GUID=00a48b7f6a4946a8adf593373e53347c;
domain=.msn.com; path=/ ; HttpOnly

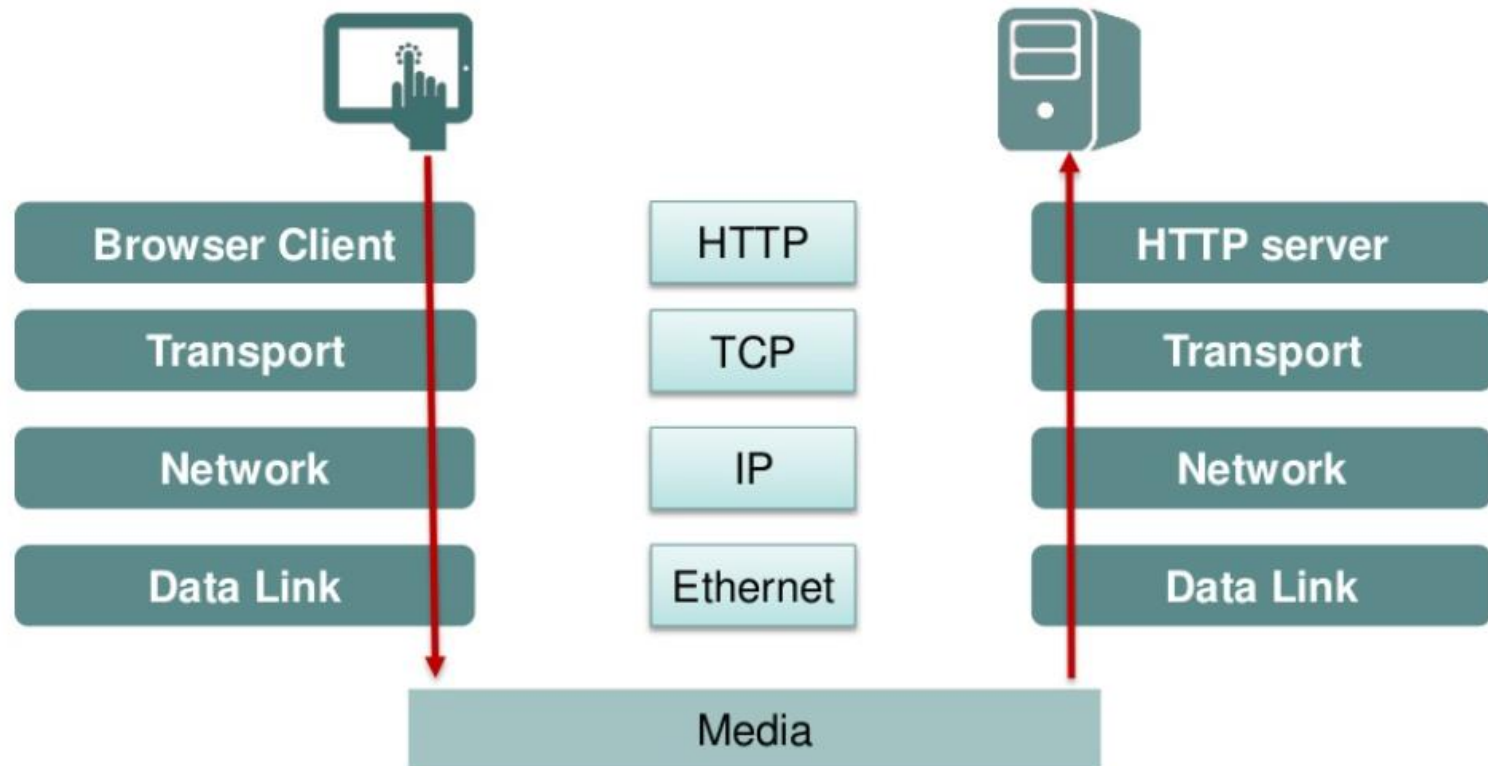
6. Disadvantages of Cookies

- They interfere with caching
 - Any response with a Set-Cookie header should not be cached, at least not the headers, since this can interfere with user identification and create security problems
- They transmit data with every request
 - Large cookie raise demand for network bandwidth
 - A cookie should never store sensitive information

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. Caching
6. Cookies
7. **Connection**
8. Security

7. Connection



Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. Caching
6. Cookies
7. Connection
8. **Security**

8. Security

- Authentication
 - Process by which a client prove its identity to the server
 - Basic
 - Digest
 - Windows
 - Form-based

8. Security: Basic Authentication

Request

GET http://localhost /demo/ HTTP/1.1
Host: localhost

Response

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="localhost"

- The WWW-Authenticate header tells the client to collect the user credentials and try again
- The realm attribute gives the user agent a string it can use as a description for the protected area
- What happens next depends on the user agent, but most browsers will display a UI for the user to enter credentials.

8. Security: Basic Authentication



Request

GET http://localhost/Demo/ HTTP/1.1

Authorization: Basic bm86aXdvdWxkbmRkb3RoYXQh

- The value of the authorization header is the client's username and password in a base 64 encoding.
- **Basic authentication is insecure by default,**

8. Security: Digest Authentication

- Digest authentication is an improvement over basic authentication because it does not transmit user passwords using base 64 encoding
- The client must send a **digest** of the password.

Request

```
GET http://localhost /demo/ HTTP/1.1  
Host: localhost
```

Response

```
HTTP/1.1 401 Unauthorized  
WWW-Authenticate: Basic realm="localhost" ,  
qop="auth,auth-int",  
nonce="dcd98b7102dd2f0e8b11d0f600bfb0c093",  
opaque="5ccc069c403ebaf9f0171e9517f40e41"
```

- **Still vulnerable to man-in-the-middle attacks in which someone is sniffing network traffic**

Digest Authentication uses MD5 cryptographic **hashing** combined with the usage of nonces to hide the password information and prevent different kinds of malicious attacks.

Digest Authentication

- ▶ So, in summary, Digest Authentication:
 - Does not send plain text passwords over the network.
 - Prevents replay attacks.
 - Guards against message tampering.
- ▶ Some of the weaknesses:
 - Vulnerability to the man-in-the-middle attacks.
 - Many of the security options are not required and thus make Digest Authentication functions in a less secure manner if not set.
 - Prevents the use of strong password hashing algorithms when storing passwords.

Due to these facts, Digest Authentication still hasn't gained major traction. Basic Authentication is much simpler, and, when combined with SSL, still more secure than Digest Authentication.

8. Security: Windows Authentication

- Windows Authentication depends on the underlying authentication protocols supported by Microsoft Windows

Request

GET http://localhost /demo/ HTTP/1.1
Host: localhost

Response

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Negotiate

- **Windows Authentication has the advantage of being secure even without using secure HTTP**
- **Require Microsoft products and servers (Active Directory)**

8. Security: Form-based Authentication

- Forms authentication is the most popular approach to user authentication over the Internet.
- It is not a standard authentication protocol and doesn't use WWW-Authenticate or Authorization headers

Request

```
GET http://localhost /demo/ HTTP/1.1  
Host: localhost
```

Response

```
HTTP/1.1 302 Found  
Location: /Login.aspx?ReturnUrl=/demo/
```

Response

```
HTTP/1.1 302 Found  
Location: /demo/  
Set-Cookie: .ASPXAUTH=9694BAB... path=/demo/; HttpOnly
```

- **Still vulnerable to session hijacking in which someone is sniffing network traffic**

Security: Authorization

- Process by which a server determines if the client has permission to use a resource

8. Security: Authorization

► 403 Forbidden HTTP status

- A web server may return a 403 Forbidden HTTP status code in response to a request from a client for a web page or resource
- Indicate that the server can be reached and understood the request, but refuses to take any further action.

```
HTTP/1.1  
403  
Forbidden
```

```
Content-Type: application/json; charset=utf-8  
Server: Microsoft-IIS/7.0  
Date: Sat, 14 Jan 2012 04:00:08 GMT  
Content-Length: 251
```

```
{  
  "code" : 123,  
  "description" : "You are not allowed to read this resource"  
}
```

42

8. Security: Encryption

- Process of transforming data so that it is unreadable by anyone who does not have a decryption key
 - Secure HTTP (TLS)

8. Security: Encryption



- **Hypertext Transfer Protocol over TLS (Transport Layer Security) is used for secure communication over a network, or perhaps more importantly – over the Internet.**
- **You would see https:// in the URI and a lock icon in the browser when you access a page that uses HTTPS.**

Fundamentals of HTTP

1. Resources
2. Request
3. Response
4. Media Type
5. Caching
6. Cookies
7. Connection
8. Security